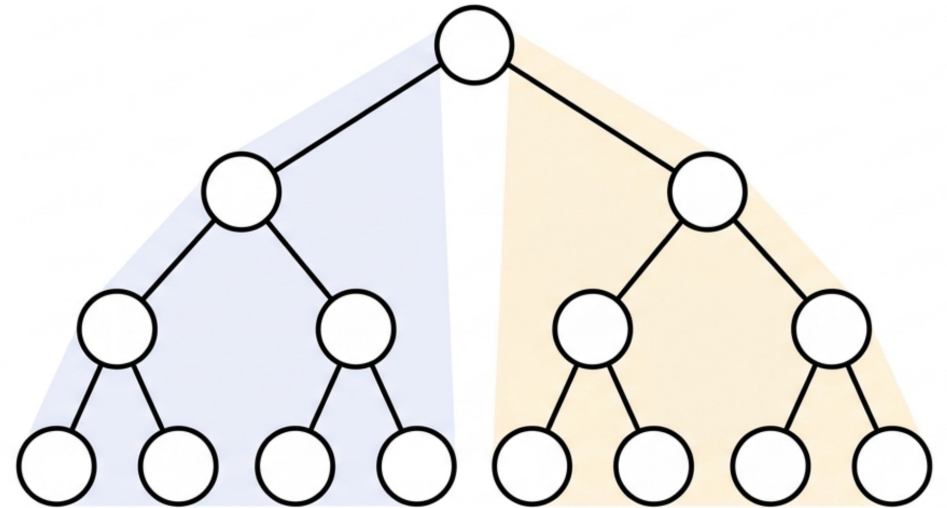




# Árboles AVL

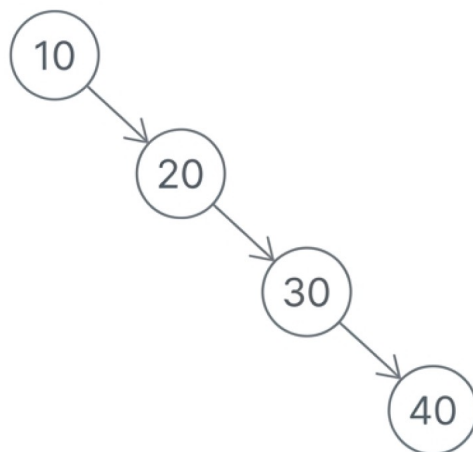


# Objetivos

- Entender la idea del factor de Equilibrio (FB) y demostrar por qué acotar la diferencia de alturas a  $\{-1, 0, 1\}$  garantiza una altura máxima de  $O(\log n)$
- Identificar, trazar algorítmicamente y codificar las cuatro rotaciones fundamentales (Rotación Simple a la Izquierda, Rotación Simple a la Derecha, Rotación Doble Izquierda-Derecha y Rotación Doble Derecha-Izquierda) necesarias para restaurar el invariante del árbol tras mutaciones en su estructura.
- Integrar la lógica de rotación y actualización de alturas en la fase de retroceso (backtracking) de la recursión durante la inserción de nodos, garantizando el balanceo sin alterar el ordenamiento del BST.
- Gestión de memoria bajo el paradigma RAI

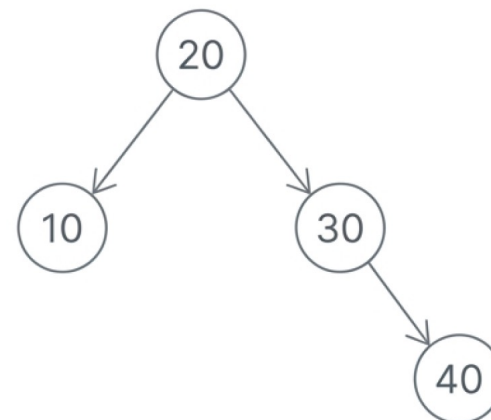
# AVL vs un BST tradicional

**Inserción secuencial (10, 20, 30, 40) en un BST estándar.**



Degeneración estructural:  
Búsqueda cae a complejidad  $O(n)$

**La misma inserción en un árbol AVL.**



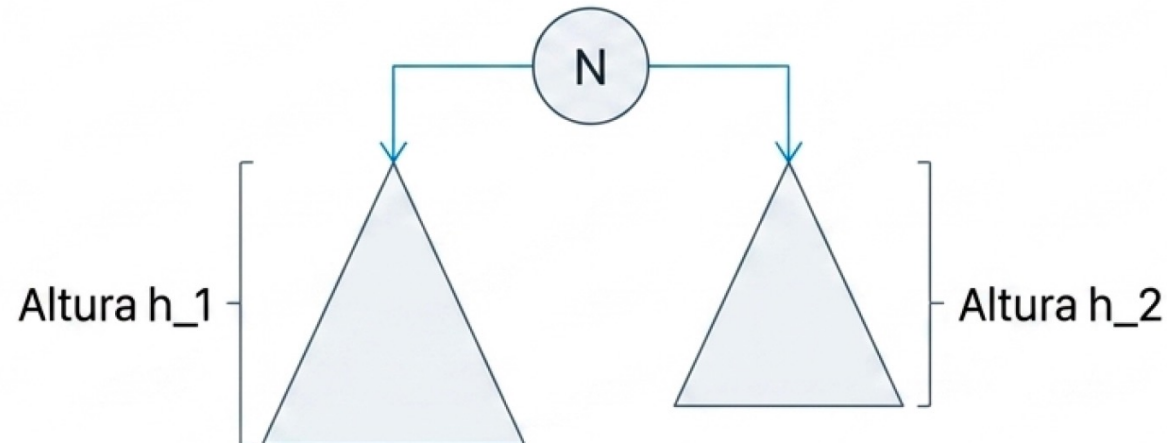
Reestructuración dinámica:  
Complejidad garantizada  $O(\log n)$

# Factor de equilibrio

En un árbol AVL (Adelson-Velsky y Landis), la diferencia de altura entre los subárboles izquierdo y derecho de CUALQUIER nodo no puede ser mayor a 1.

**Factor de Equilibrio** = Altura(Izquierdo) - Altura(Derecho)

$$|h_1 - h_2| \leq 1$$



# Factor de equilibrio

```
/** @brief Obtiene la altura de forma segura (nulo = 0). */
int height(const std::unique_ptr<AVLNode>& node) const {
    return node ? node->height : 0;
}

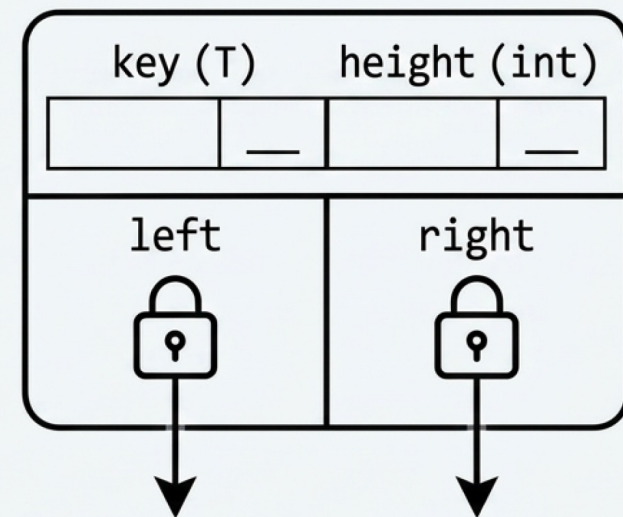
/** @brief Calcula el Factor de Equilibrio:  $FB = h(izq) - h(der)$ . */
int getBalance(const std::unique_ptr<AVLNode>& node) const {
    return node ? height(node->left) - height(node->right) : 0;
}
```

# Nodo AVL

La estructura **AVLNode** encapsula la altura y utiliza `std::unique_ptr` para poseer recursivamente a sus hijos.

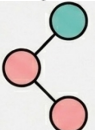
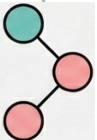
```
template <typename T>
struct AVLNode {
    T key;
    int height = 1;
    std::unique_ptr<AVLNode<T>> left;
    std::unique_ptr<AVLNode<T>> right;

    explicit AVLNode(T k) : key(k) {}
};
```



Nota: Al aislar la propiedad en `unique_ptr`, el destructor por defecto limpia todo el subárbol automáticamente en cascada ( $O(n)$  seguro).

# Desbalances

|              |                            | Hijo                                                                                                                                         |                                                                                                                                                     |
|--------------|----------------------------|----------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------|
|              |                            | FB = +1 (Alineado)                                                                                                                           | FB = -1 (Zig-Zag)                                                                                                                                   |
| Nodo Crítico | FB = +2 (Izquierda pesada) |  <p>Caso LL -&gt; <b>Rotación Simple a la Derecha</b></p>   | <p>Caso LR -&gt; <b>Rotación Doble (Izquierda-Derecha)</b></p>   |
|              | FB = -2 (Derecha pesada)   |  <p>Caso RR -&gt; <b>Rotación Simple a la Izquierda</b></p> | <p>Caso RL -&gt; <b>Rotación Doble (Derecha-Izquierda)</b></p>  |

Cada desbalance local cae inequívocamente en una de estas cuatro configuraciones.

# Rotaciones simples

Soluciona escenarios lineales (Casos LL y RR). La rotación mueve el nodo desequilibrado hacia abajo, elevando a su hijo para que asuma como nueva raíz del subárbol.





# Rotaciones simples

```
template <typename T>
std::unique_ptr<AVLNode<T>> rightRotate(std::unique_ptr<AVLNode<T>> y) {
    // Paso 1: Transferencia de propiedad de los nodos involucrados
    // x 'se adueña' del hijo izquierdo de y.
    std::unique_ptr<AVLNode<T>> x = std::move(y->left);
    // T2 'se adueña' del hijo derecho de x (subárbol que cambiará de padre).
    std::unique_ptr<AVLNode<T>> T2 = std::move(x->right);

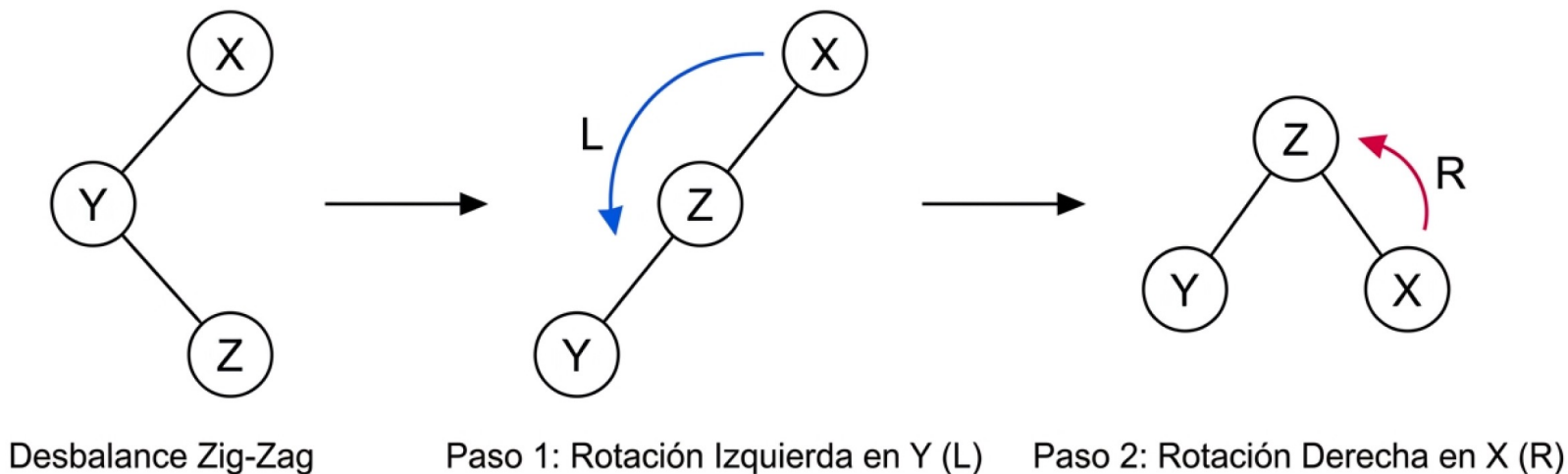
    // Paso 2: Reestructuración topológica (Rotación)
    // El subárbol T2 se vincula como hijo izquierdo de y.
    y->left = std::move(T2);
    // y se vincula como hijo derecho de x. x ahora posee a y.
    x->right = std::move(y);

    // Paso 3: Actualización de alturas (Crucial: Orden descendente)
    // Primero actualizamos 'y' porque ahora es hijo de 'x'. Su altura depende de sus nuevos hijos.
    x->right->height = std::max(height(x->right->left), height(x->right->right)) + 1;
    // Finalmente actualizamos 'x', que es la nueva raíz del subárbol.
    x->height = std::max(height(x->left), height(x->right)) + 1;

    // Paso 4: Retornamos el nuevo nodo dominante del subárbol
    return x;
}
```

# Rotaciones dobles

Cuando la inserción crea una forma de zig-zag (Casos LR y RL), una rotación simple transfiere el problema en lugar de resolverlo. Requiere una composición de dos pasos.

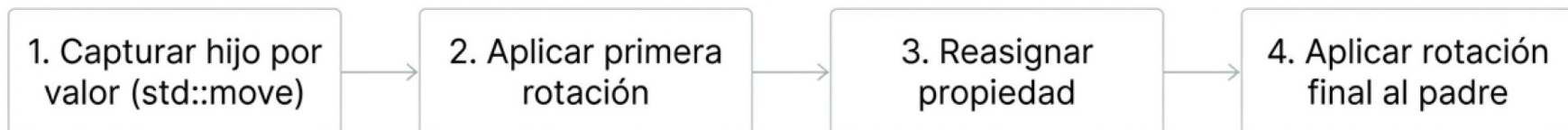


**Fase 1:** Rotación inferior en dirección opuesta al desequilibrio (alinea los nodos en línea recta).

**Fase 2:** Rotación superior sobre la raíz original (restaura el equilibrio total).

# Rotaciones dobles

```
// Caso 3: Izquierda-Derecha (LR) - El desbalance está en el hijo interno.  
// Se soluciona con una rotación doble: primero izquierda al hijo, luego derecha a la raíz.  
if (balance > 1 && key > node->left->data) {  
    node->left = leftRotate(std::move(node->left));  
    return rightRotate(std::move(node));  
}  
  
// Caso 4: Derecha-Izquierda (RL) - El desbalance está en el hijo interno.  
// Se soluciona con una rotación doble: primero derecha al hijo, luego izquierda a la raíz.  
if (balance < -1 && key < node->right->data) {  
    node->right = rightRotate(std::move(node->right));  
    return leftRotate(std::move(node));  
}
```



# Inserción

La inserción delega la asignación de memoria a `std::make_unique`, previniendo fugas incluso si ocurren excepciones.

```
/**
 * @brief Inserción con balanceo.
 * Utiliza semántica de movimiento para manejar la recursión con unique_ptr.
 */
std::unique_ptr<AVLNode> insertRec(std::unique_ptr<AVLNode> node, const T& key, bool& inserted) {
    if (!node) {
        inserted = true;
        return std::make_unique<AVLNode>(key);
    }

    if (key < node->data) {
        node->left = insertRec(std::move(node->left), key, inserted);
    } else if (key > node->data) {
        node->right = insertRec(std::move(node->right), key, inserted);
    } else {
        return node; // Duplicado detectado: no se inserta.
    }
}
```

# Inserción

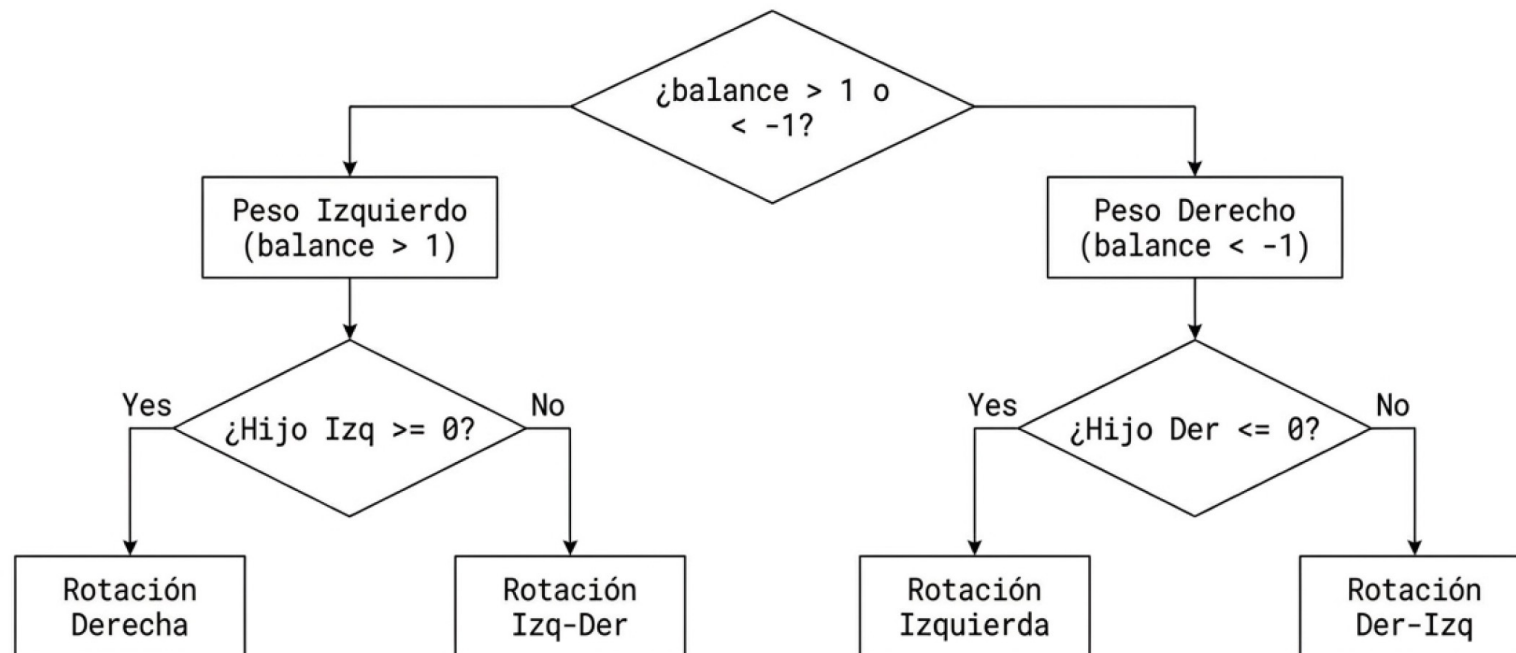
Al regresar de las llamadas recursivas (ascendiendo hacia la raíz) verificamos el equilibrio de cada nodo

```
// 1. Actualización post-recursión (Backtracking)
node->height = 1 + std::max(height(node->left), height(node->right));
int balance = getBalance(node);

// 2. Corrección de los 4 casos de desbalanceo
// Caso LL
if (balance > 1 && key < node->left->data) return rightRotate(std::move(node));
// Caso RR
if (balance < -1 && key > node->right->data) return leftRotate(std::move(node));
// Caso LR (Rotación Doble)
if (balance > 1 && key > node->left->data) {
    node->left = leftRotate(std::move(node->left));
    return rightRotate(std::move(node));
}
// Caso RL (Rotación Doble)
if (balance < -1 && key < node->right->data) {
    node->right = rightRotate(std::move(node->right));
    return leftRotate(std::move(node));
}

return node;
}
```

# Inserción



# Rendimiento de un árbol AVL

La estricta gestión del Factor de Equilibrio garantiza matemáticamente que el árbol nunca degenerará en una estructura lineal, asegurando eficiencia logarítmica.

| Operación            | BST<br>(Caso Promedio) | BST<br>(Peor Caso) | Árbol AVL<br>(Todos los Casos) |
|----------------------|------------------------|--------------------|--------------------------------|
| Búsqueda (find)      | $O(\log n)$            | $O(n)$             | $O(\log n)$                    |
| Inserción (insert)   | $O(\log n)$            | $O(n)$             | $O(\log n)$                    |
| Eliminación (remove) | $O(\log n)$            | $O(n)$             | $O(\log n)$                    |

El costo adicional de las rotaciones  $O(1)$  en memoria es trivial comparado con la prevención del peor caso  $O(n)$ .